GROUP 94 / FINAL SUBMISSION RELEASE

Campus Pantry FoodHub

Completed changes and validation report

Prepared for Ziyue Wu, Nikhil Khatri and **Samowel Aboud**
24 October 2026

REPEATABLE TESTS	CONCURRENCY	PRESERVED DESIGN
44/44 passed in each of two runs	10 coordinated race scenarios	16 tables 3 reporting views

Release outcome

The supplied Python CLI and PostgreSQL project has been revised for the requested **university submission and showcase**. The release fixes the identified data-integrity and workflow defects, supplies safe setup and testing, and includes a repeatable demonstration that works after hours.

The full showcase completed booking, duplicate rejection, volunteer approval, attendance, stock reconciliation and shift closure. Its stock example checks **100 + 10 - 2 - 88 = 20** estimated units dispensed. Both final test databases and the demonstration database were removed after verification.

What this report records

Pages 2-5 explain all sixteen completed change groups and why they were needed. Pages 6-7 give the executed evidence, setup entrypoints, and the remaining boundaries of the selected educational scope.

Release boundary

This is a tested local showcase release, with synthetic identities and simulated UniKey sign-in. It does not claim public deployment readiness, real university authentication, email delivery, QR image generation, a security certification or a measured load capacity.

Source baseline: Foodhub_project.zip supplied by the user. The original archive was not overwritten. Revised source and evidence are packaged separately.

DATA RULES

Attendance and relational integrity

C04 - Scheduling and staffing integrity

Completed: shift/session schedules and session parent linkage are immutable; capacity cannot be lowered below current noncancelled bookings. Staff changes are allowed only while PLANNED, preserving required staffing after opening. Repeated same-key assignment and valid role correction while PLANNED work without false overlap errors. **Why:** parent edits and assignment deletion must not invalidate previously valid child data. **Files:** 01_schema.sql, services.py.

C05 - Booking-date attendance and late arrivals

Before: an old confirmed token could check in on a later date. **Completed:** check booking date and the 10:00 inclusive-16:00 exclusive Sydney window before idempotence; reject unopened shifts. Accept a same-day late arrival even after its original shift has CLOSED, preserving the supplied operating policy. Only mark no-shows at/after day end. **Why:** prevent stale attendance without penalising permitted late arrivals. **Files:** services.py, tests/test_release.py.

C06 - Audited overrides and attendance metadata

Before: an early trigger return could bypass override validation on UPDATE. **Completed:** always validate override actor/reason consistency; ordinary bookings cannot retain override audit fields. Overrides do not bypass capacity. Attendance timestamp exists if and only if status is ATTENDED. **Why:** updates must satisfy the same rules as inserts and leave consistent evidence. **Files:** 01_schema.sql, services.py.

C08 - Purchase/batch relationship

Before: changing a purchase line's product could leave a batch pointing to the wrong product. **Completed:** composite foreign key (purchase_line_id, product_id) references a matching unique purchase-line key. **Why:** declarative referential integrity covers both parent and child updates. **File:** 01_schema.sql.

BUSINESS CORRECTNESS

Stock and operational workflows

C01 - Genuine stock observations

Before: recording a closing count first invented an equal opening; an adjustment could invent a zero opening. **Completed:** require a real opening before either action; reject duplicate openings/closings; commit the observed count and summary together. **Why:** inferred dispensing must not be based on fabricated observations. **Files:** services.py, tests/test_release.py.

C02 - Reconciliation and closed inventory

Completed: positive whole-number adjustments require a reason, cannot exceed available stock as waste, and cannot follow a closing count. Closing cannot exceed opening plus receipts minus waste. A shift cannot close with an unfinished count or negative estimated dispensing. Database triggers protect closed stock records against INSERT/UPDATE/DELETE. **Why:** prevent impossible balances and preserve completed shift stock reports. **Files:** services.py, 01_schema.sql.

C03 - Complete shift workflow

Before: the menu created PLANNED shifts but offered no supported way to open/close them. **Completed:** staff actions for PLANNED -> OPEN -> CLOSED, active staffing checks, upcoming-shift listing and per-shift reconciliation reporting. Reopening is rejected. **Why:** the full operating workflow is now possible without manual SQL. **Files:** cli.py, services.py, 01_schema.sql.

C09 - Active roles and volunteer decisions

Completed: hide inactive roles and require an active acting staff member for privileged mutations/reports; validate volunteers at service entry. Review only a PENDING request, rejecting stale decisions, ended shifts and closed shifts. **Why:** inactive identities cannot continue through an old portal, and one reviewer cannot overwrite another's decision. **Files:** services.py, cli.py. UniKey selection remains simulated authentication.

RUNTIME RELIABILITY

Transactions, input and installation

C07 - Concurrency and transaction order

Completed: consistent student-then-session locks for booking; shared staff/volunteer row locks before overlap decisions; ordered affected-record locks; shift locks coordinate closing and inventory. Queries and writes use transaction boundaries with rollback and limited retries for PostgreSQL serialization/deadlock errors. **Why:** simultaneous requests must not both claim the last place or overlapping duty. **Files:** 01_schema.sql, services.py. Bounded race tests demonstrate targeted cases, not arbitrary load capacity.

C10 - Recoverable input and menu errors

Before: a malformed volunteer-filter date could escape the menu's error handler; an invalid staffing choice defaulted to assistant. **Completed:** strict calendar-date parsing, consistent database error translation and rollback, explicit menu-choice validation, friendly missing-schema/connection errors, and upcoming filters that omit ended shifts. **Why:** ordinary input errors must leave the menu and transaction usable. **Files:** cli.py, services.py, db.py.

C11 - Predictable connections and targeted indexes

Completed: one configured endpoint, no silent port fallback, five-second default connection timeout and bounded database waits. Pinned psycopg2-binary 2.9.10. Added/revised indexes for latest-count ordering and staff/volunteer overlap lookups. **Why:** configuration failures are explainable and repeated lookup predicates have suitable indexes. **Files:** db.py, requirements.txt, 01_schema.sql. No measured throughput or latency improvement is claimed; no connection pool was added.

C12 - Canonical, nondestructive setup

Before: legacy patches could overwrite the corrected booking trigger and rebuild scripts could destroy data. **Completed:** canonical schema/seed with ON_ERROR_STOP wrappers; existing schema makes initialization fail without dropping data. Legacy patches emit no-op notices. Seed loading follows the revised lifecycle. **Why:** a fresh installation always receives the same rules and accidental reruns preserve existing rows. **Files:** campus_pantry_sql and campus_pantry_app/sql. This is not a migration for an existing deployment.

REVIEWABLE HANDOVER

Tests, demonstration and documentation

C13 - Isolated verification and safe cleanup

Before: tests used broad identifiers/prefixes for cleanup and could mutate seed/application records. **Completed:** explicit test target settings, random database names, ownership markers checked before teardown, per-test fixture reset only inside the owned database, and nonzero status if cleanup fails. **Why:** verification must not damage someone else's data. **Files:** tests/support.py, tests/run_tests.py; legacy test entrypoints now delegate safely.

C14 - Assertions that prove their outcome

Before: some tests printed PASS in both branches, accepted any exception, used fixed dates or only executed analytics. **Completed:** named unittest assertions with expected errors and values, barrier-coordinated races with separate connections/timeouts, current-day fixtures and explicit clock-boundary tests. Validate all twelve query groups/thirteen result sets. Replace SQL failure tests with 23 specific assertions and compare all table rows before/after rollback scripts. **Why:** a green result now checks the intended rule, not merely absence of a crash. **Files:** tests, 04_transactions.sql, 05_validation.sql, 06_failure_tests.sql.

C15 - Repeatable after-hours demonstration

Completed: showcase.py runs the real services in a disposable database at a clearly labelled simulated business time. It verifies booking, duplicate rejection, volunteer approval, attendance, shift closure and a 20-unit reconciliation, then proves cleanup. **Why:** the presentation works outside operating hours without weakening the normal CLI clock rules. **Files:** showcase.py, tests/support.py.

C16 - Accurate handover and evidence

Completed: rewritten setup/use instructions, change report, captured test results and demo transcript, clear local-showcase scope, synthetic identities and AI acknowledgement. Removed old misleading documentation from the release package; did not reissue signatures or participation claims. **Why:** reviewers can reproduce results and distinguish implemented functionality from simulated integrations. No Canvas submission, email delivery or public deployment has occurred.

EVIDENCE, NOT ASSUMED RESULTS

Executed validation

Two complete runs of the final **44-test** suite passed against actual **PostgreSQL 16.15**. Each run created a different database and verified its removal. There were **zero failures, errors or skipped tests**.

Evidence	Observed result
Final run 1	44 passed; cleanup verified
Final run 2	44 passed; cleanup verified
Concurrency	10 synchronized separate-connection scenarios in each run
SQL assertions	23 specific expected-failure checks; all table rows preserved after rollback
Analytics	12 named groups / 13 SELECT result sets checked against known values
Showcase	Full service workflow passed; 20-unit reconciliation; cleanup verified
Setup wrappers	First run succeeds; repeat refuses existing schema and preserves 29 seed people

Coverage that matters

Booking: exact 50-seat boundary, final-seat contention, same-student daily collisions, override updates, cancellation and mixed SQL/service access. Rosters: direct SQL and service overlap checks, competing reviewers, repeated assignments and role correction. Attendance: old/future tokens, opening/closing boundaries, repeated scans, no-shows and same-day late arrivals after shift closure.

Inventory and runtime: missing/duplicate observations, concurrent openings, exact reconciliation, closed-history edits, parent key/capacity edits, malformed dates, invalid staffing choices, active-role checks and transaction rollback. Tests require the intended error; a timeout or unrelated exception is not a passing business-rule result.

Evidence files in the ZIP

evidence/test-results-run-1.json and test-results-run-2.json contain individual results and runtime metadata. The matching text logs contain the unittest output. docs/showcase-transcript.txt captures the end-to-end demonstration. docs/VALIDATION.md explains reproduction and limits.

Tests ran on Windows with Python 3.12 and psycopg2-binary 2.9.10. This is targeted regression evidence, not a security audit, arbitrary-schedule concurrency proof, load test or test of a hosted production deployment.

HANDOVER

Run and present the release

Start with README.md

Extract Foodhub_Group94_Canonical_Submission.zip and open START_HERE.md. Install the pinned requirements in a virtual environment and configure your local PostgreSQL credentials. README.md provides PowerShell instructions and macOS/Linux equivalents. The server and Python dependency are prerequisites; no database binaries or credentials are bundled.

```
python showcase.py
python tests/run_tests.py
python campus_pantry_app/cli.py
```

The first two commands use explicit TEST_PG target settings and a database role allowed to create databases. They create and remove their own databases. The interactive CLI uses normal PG settings and a separately initialized database. Never apply the original legacy SQL patches to the revised release.

Suggested showcase narrative

1. Explain Person and the separate student, staff and volunteer roles.
2. Run the isolated showcase and show the duplicate-booking rejection.
3. Explain why attendance is separate from booking and allows same-day late arrivals.
4. Show the 20-unit stock calculation, then the final test evidence.
5. Explain the shared locks, transactions and constraints that protect the data.

Design boundaries to explain accurately

UniKey selection identifies a simulated role; it does not verify a real person. Staff service checks and the database constraints are complementary, and direct database owners can bypass service-only time and identity policies. Tokens are terminal strings, not delivered email or rendered QR images.

Stock dispensing is inferred from physical counts. Adjustments aggregate by shift, batch and kind and retain the latest reason; this is not a complete event ledger. A shift with no inventory observations can close. Reopening is deliberately disallowed, and scheduling changes need replacement records. This fresh-install package does not migrate an existing database.

Acknowledgement and responsibility

OpenAI Codex assisted with review, implementation, tests and this documentation. Group members should review and understand the changes and acknowledge AI assistance under the unit requirements. The group remains responsible for submitted work. No Canvas submission, email, public deployment, signature or participation attestation was made as part of this revision.

Document basis: supplied source archive and source workflow material; revised code, executed PostgreSQL results and captured demo output. No unsupported speedup, authentication or production claim is used.